Student AI Financial Ecosystem

Product Requirements Document

Community savings + agentic spending + partner rewards + Razorpay execution

Version 1.0 | Hackathon scope | India-first student product

Executive product statement: A student-first financial ecosystem where users build a small emergency-oriented savings balance through a community savings pool inspired by chit funds, receive extra incentives from milestones and partner offers, and use an agentic spending experience governed by explicit budgets and rules. Razorpay is the payment execution rail for the prototype; the AI is never allowed to bypass deterministic financial rules.

IMPORTANT SCOPE GUARDRAIL: This PRD defines a hackathon prototype. It does not define a live chit-fund, deposit-taking, lending, investment-advice, or card-issuing business. Real-money custody, regulated investment products, and live card issuance require separately validated legal/compliance/partner decisions.

1. Product at a Glance

Primary user: Indian college student who wants to save small amounts, maintain an emergency cushion, get useful discounts, and control discretionary spending without managing a complex financial app.

Area	Product decision	Hackathon status
Savings	₹100–₹500 recurring contributions; emergency-oriented goal	Core
Community pool	Virtual pool inspired by chit mechanics; individual balances remain distinct	Core demo / simulated economics
Rewards	Streaks, milestones, cashback/partner-funded offers	Core
Agentic spending	Rule-bound AI spending assistant; no uncontrolled credit	Core
Razorpay	Test-mode payment execution + webhook confirmation; RazorpayX test payouts where needed	Core integration
Investment returns	Not part of MVP live money flow; can be simulated as a future/partner layer	Out of MVP

2. Problem

Students often have small, irregular cash flows and weak saving habits. Existing finance products can be too investment-heavy, too generic, or too passive. The product should turn saving into a visible habit, create a community incentive loop, and make spending limits actionable through an AI agent.

3. Core Value Proposition

“Save a small amount, build a cushion, unlock community and merchant rewards, and let your AI enforce the spending rules you choose.”

User need	Product response	Success signal

Build emergency cushion	Recurring micro-contributions + goal tracking	Savings consistency
Need short-term liquidity	Community-pool inspired early-access mechanism (simulated)	Liquidity requests resolved transparently
Want immediate motivation	Streaks + milestones + partner rewards	Reward redemption / retention
Control spending	Agentic rules + approvals + hard limits	Unauthorized actions blocked
Want relevant deals	AI matches user needs to partner offers	Offer engagement / savings

4. Product Scope

MVP user journey

```
Onboard → set goal + monthly limit → contribute → join pool → earn reward eligibility →
agent monitors spending → user asks/authorizes purchase → policy check → Razorpay test payment →
webhook confirmation → update balance/reward/audit trail → next cycle adapts.
```

4.1 Community Pool — Product Rules

Use the chit-fund concept as a user experience and incentive mechanism, not as an unqualified claim that the app itself is a legally operated chit fund.

- Example demo: 10 students contribute ₹500 each → virtual cycle amount ₹5,000.
- A participant can request an early liquidity payout/benefit based on transparent cycle rules.
- The difference/discount can be represented as a reward/dividend pool in the prototype.
- Each user has an individual ledger; do not model user money as an unrestricted common bank balance.
- Every pool allocation must be explainable: contribution, rule applied, reward/discount, resulting user balance.

4.2 Rewards + Ads/Partner Offers

The platform should prioritize useful student offers rather than generic advertising. Offers can include fashion, electronics, food, subscriptions, travel, and campus services. In the prototype, offers are curated synthetic partner offers with explicit merchant-funded reward economics.

- Example: “₹2,000 headphones, partner discount ₹200” or “10% off fashion order.”
- Agent should rank offers by user need, price, eligibility, and expected value—not merely highest advertiser payment.
- Reward source must be explicit in the data model: platform-funded, partner-funded, or pool-funded.
- Never present a promotional recommendation as neutral financial advice.

4.3 Agentic Credit-Card / Spending Experience

Product interpretation: an agentic spending layer inspired by agentic-card concepts, not a claim that the prototype is issuing a real credit card. The UI should look/feel like a controlled spending account with rules, limits, optional approvals, and purchase execution.

User rule	Agent behavior	Hard outcome
₹1,000/month discretionary limit	Track committed + completed spend	Block above limit
Never touch emergency savings	Treat emergency balance as protected	Block attempt
Purchases above ₹500 need approval	Request approval before execution	No payment without approval
Fashion offers preferred	Use offer-ranking tool when shopping	Recommend best eligible offer
Pause spending	Stop new agentic purchase actions	No action until resumed

5. Agentic Architecture — Exact Definition

Recommended architecture: ONE primary LLM Financial Agent with deterministic domain tools/engines and a mandatory policy/guardrail layer. Do not deploy three independent LLM agents for Savings, Pool, and Card in the MVP. Multiple LLMs create coordination and consistency risk without adding useful accuracy for this scope.

5.1 One Agent, Three Capability Domains

Capability	What the agent can reason about	Allowed actions
Savings	Goal, recent cash flow, contribution history, safe contribution recommendation	Recommend contribution; request payment creation only within rule limit
Community Pool	Cycle state, contribution status, eligibility, reward/discount rule	Explain eligibility; create pool event/record; request simulated allocation/payout
Spending + Offers	Budget used, remaining limit, purchase intent, merchant offers, approval policy	Recommend product; request purchase; apply eligible offer; stop/ask approval

5.2 The Agent Loop

```
1. Observe: fetch current user state and relevant records.
2. Interpret: convert user intent into a structured task.
3. Plan: choose the minimum tools needed.
4. Policy check: validate amount, purpose, authorization, limits, and state.
5. Act: call a permitted tool; only backend services can touch Razorpay.
6. Verify: wait for tool result/webhook or fetch authoritative status.
7. Reconcile: update ledger, goal, reward, and audit record.
8. Respond: explain what happened and what remains pending.
9. Adapt: next recommendation uses the updated state.
```

5.3 Agent Tools — Bounded Tool Contract

Tool	Input	Output	LLM may call?
get_user_profile	user_id	profile, rules, flags	Yes
get_wallet_or_ledger	user_id	balances, reserved amounts, recent events	Yes
get_transactions	user_id, period	categorized transaction summary	Yes
calculate_safe_contribution	user_id, goal_id	recommended amount + reasons	Yes
get_pool_status	pool_id	cycle, eligibility, contribution state	Yes
get_eligible_rewards	user_id, context	ranked eligible rewards	Yes
get_offers	intent, category, budget	eligible offers	Yes
check_policy	action, amount, purpose	ALLOW / DENY / REQUIRE_APPROVAL + reason	Yes, mandatory before action
create_payment_intent	amount, purpose, user_id	internal intent id	Only after policy allows
create_razorpay_payment	intent_id	Razorpay reference / error	Backend only
get_payment_status	payment_id	authoritative status	Yes
process_test_payout	recipient, amount, reason	payout reference / status	Backend only, policy-gated
update_goal	goal_id, event	new goal state	Yes
write_audit_event	event	audit id	System service

5.4 What the LLM Must NEVER Do

- Never invent balances, transaction values, payment statuses, rewards, offers, or pool allocations.
- Never directly call a raw money-moving endpoint; all money actions go through a backend action service and policy check.
- Never override a denied policy result because a user asks repeatedly.
- Never infer authorization from conversation alone when a structured user rule/approval is required.

- Never fabricate a Razorpay success response; success must come from the payment API/webhook/status check.
- Never claim live credit-card issuance, investment returns, or regulated investment custody in the prototype.

5.5 State Machine for Financial Actions

```
PROPOSED → POLICY_CHECK →
  DENIED → CLOSED
  NEEDS_APPROVAL → AWAITING_APPROVAL → APPROVED → EXECUTING
  ALLOWED → EXECUTING
EXECUTING →
  SUCCESS → VERIFIED → LEDGER_UPDATED
  FAILURE → RECOVERY/RETRY_POLICY → CLOSED
  UNKNOWN → RECONCILE_STATUS → VERIFIED or EXCEPTION
```

6. Guardrails and Accuracy

The system should achieve accuracy through layered controls, not by asking the LLM to “be careful.” The deterministic policy layer is the source of truth for financial permissions.

Layer	Responsibility	Failure behavior
Schema validation	Reject malformed tool arguments	Do not execute
Policy engine	Limits, purpose, approvals, protected balances	DENY or REQUIRE_APPROVAL
Authorization state	Check user-approved capability and expiry	DENY
Idempotency	Prevent duplicate payment/payout execution	Return existing intent/result
Razorpay status	Authoritative payment/payout state	Reconcile before marking success
Webhook validation	Signature + event processing	Reject invalid event
Audit log	Record proposal, policy result, action, result	Every money action traceable
Exception queue	Handle unknown/ambiguous states	Human/manual review in demo

6.1 Demo Evaluation Bar

Use a synthetic benchmark of at least 100 scenarios to prove behavior across normal, overspending, insufficient balance, duplicate requests, changed goals, unusual spending, unauthorized amounts, failed payments, and reward eligibility edge cases.

Metric	Target
Policy compliance	≥95%
Correct financial decisions on benchmark	≥90%
Unauthorized action blocking	100%
Duplicate execution prevention	100% in tested duplicate cases
Payment status correctness	100% on simulated/webhook test cases
Honest exception reporting	Every unsupported/ambiguous case surfaced

7. Razorpay Integration (Prototype)

Current verified scope: Razorpay Test Mode is a sandbox; test transactions use dummy data and no real money. Separate Test and Live keys are used, and test keys begin with rzp_test_. Razorpay recommends webhooks for event notifications.

Verified documentation references: Razorpay Quickstart and Test/Live Modes; Payments API; Webhook validation/testing; RazorpayX Test Mode and Payouts API.

https://razorpay.com/docs/payments/quickstart/

https://razorpay.com/docs/payments/dashboard/test-live-modes/

https://razorpay.com/docs/api/payments/

https://razorpay.com/docs/webhooks/validate-test/

https://razorpay.com/docs/x/dashboard/test-mode/

https://razorpay.com/docs/api/x/payouts/

https://razorpay.com/agentic-payments/

7.1 Payment Flow

```
User intent → Agent creates structured action → Policy Engine → internal payment_intent →
Razorpay Test Mode payment/order flow → webhook received → signature validated → status reconciled →
ledger + goal + reward updated → user notified.
```

- Do not treat a frontend callback as the final source of truth for a money status.
- Handle duplicate and out-of-order webhook deliveries idempotently.
- For RazorpayX Test Mode payouts, model Contact → Fund Account → Payout as required by the API flow; test-mode payouts use dummy balance and have test-specific lifecycle behavior.
- Use idempotency for payouts; Razorpay documents the idempotency key as mandatory for payout requests starting March 15, 2025.

7.2 Agentic Payment Positioning

Razorpay currently describes UPI Reserve Pay as consent-based, pre-authorized payments within approved spending limits. For this hackathon, the prototype can demonstrate the same product principle—explicit user authorization + bounded spending—even if the demo uses ordinary Test Mode payment APIs rather than a production agentic payment rail.

8. Instructions for Coding Agents

This section is the operational source of truth for coding agents. Coding agents should implement only behavior explicitly defined here or in linked repository specifications. When a requirement is missing, they must ask or create a clearly marked TODO rather than inventing a financial rule.

8.1 Source-of-Truth Hierarchy

1. Product requirement in this PRD.
2. Structured schemas / enums in the codebase.
3. Deterministic policy configuration.
4. External API documentation supplied by the project.
5. LLM output is NEVER a source of truth for financial state.

8.2 Coding-Agent Non-Hallucination Rules

- Do not invent a new fee, interest rate, pool formula, reward amount, credit limit, card feature, or merchant offer.
- Do not create a tool unless its purpose, input, output, and permission are defined.
- Do not allow an LLM-generated amount to bypass `check_policy`.
- Do not mark a transaction successful without verified provider state.
- Do not add autonomous lending, investing, custody, KYC/AML claims, or real card issuing unless the PRD is explicitly updated.
- All synthetic data must be labeled synthetic/demo data.
- Prefer deterministic calculations for money, percentages, limits, balances, eligibility, and reward math.

8.3 Recommended Backend Modules

Module	Purpose
agent_orchestrator	LLM planning, tool selection, structured response
policy_engine	Hard limits + approval logic
money_action_service	Creates internal action intents; invokes provider only

	after policy
razorpay_adapter	Encapsulates Razorpay API calls; no LLM access to credentials
webhook_service	Signature validation, idempotency, event reconciliation
ledger_service	Append-only financial event model + derived balances
pool_service	Cycle, contribution, allocation, reward/dividend simulation
reward_service	Milestones, streaks, merchant-funded rewards
offer_service	Offer catalog, eligibility, ranking
audit_service	Every agent decision and financial action trace
benchmark_service	100+ scenario test harness and metrics

8.4 Minimal Domain Objects

Entity	Key fields
User	id, profile, goals, limits, approval_policy, status
Goal	id, target_amount, current_amount, cadence, status
LedgerEvent	id, user_id, type, amount, source, status, timestamp
PoolCycle	id, size, contribution_amount, members, rules, status
PoolAllocation	cycle_id, user_id, amount, reason, status
Reward	id, user_id, source, amount/value, eligibility, status
Offer	id, merchant, category, discount, expiry, funding_source, eligibility
SpendPolicy	monthly_limit, per_tx_limit, approval_threshold, protected_buckets
ActionIntent	id, user_id, type, amount, purpose, policy_result, provider_ref, status
AuditEvent	actor, action, inputs_hash, policy_result, provider_result, timestamp

9. Representative End-User Behaviors

```
A. “Save ₹500 this month.”
Agent checks goal + recent spend → recommends/accepts ₹500 → policy allows → payment intent → Razorpay → webhook → goal updated.

B. “Buy these headphones if they are under ₹2,000 and I have the student discount.”
Agent finds eligible offer → checks budget + approval rule → asks approval if required → payment → verify → record reward.

C. “Spend whatever you need from my emergency fund.”
Agent refuses autonomous access because emergency funds are protected → explains rule → offers a manual/approved path.

D. “Pay ₹800 again.”
Agent checks existing action intent → blocks duplicate if already completed/processing → no second charge.
```

10. Failure Handling

Failure	Expected UX
Payment failed	“Payment failed. Your savings goal was not increased.” Keep ledger consistent.
Webhook delayed	Show “processing”; reconcile via provider status; do not claim success.
Duplicate request	Explain existing payment/action; do not create duplicate.
Insufficient test balance	Show failed/queued behavior according to provider; no false success.
Offer expired	Hide or mark unavailable; never fabricate discount.
Pool rule ambiguous	Do not guess allocation; create exception/manual review state.
LLM produces invalid tool args	Schema validation rejects call; agent retries with

	corrected structured arguments or asks user.

11. Out of Scope for Hackathon MVP

- Real-money pooled deposits or real chit-fund operation.
- Real credit-card issuance, credit underwriting, interest charging, or revolving credit.
- Autonomous investment management or promises of financial returns.
- Live custody/holding of customer funds without a regulated partner design.
- Production KYC/AML workflows.
- Using scraping or invented merchant data for “real” offers.
- LLM-only financial calculations.
- Multi-agent orchestration unless later justified by benchmark evidence.

12. MVP Acceptance Criteria

- A user can create a savings goal and contribution rule.
- A user can join a demo community pool and see cycle state, contributions, and transparent reward/discount logic.
- A user can see personalized synthetic partner offers.
- A user can define spending limits and approval thresholds.
- The single Financial Agent can observe state, plan, call tools, execute bounded actions, verify results, and adapt its next step.
- Every payment action passes through policy checks and produces an audit record.
- Razorpay Test Mode payment integration is functional; webhook verification/reconciliation works.
- At least 100 synthetic scenarios are executed and benchmark metrics are reported.
- Any unsupported or ambiguous case is surfaced as an exception rather than hallucinated.

13. Open Decisions Before Production

- Whether the community mechanism will ever use real pooled money and under what legal/regulated structure.
- Which regulated partner would hold/invest emergency savings, if investment returns are introduced.
- Whether a real card product is needed or whether an agentic spending account is sufficient.
- Commercial contracts and economics for merchant-funded rewards.
- Consent, privacy, and data-source model for transaction enrichment.
- Live-mode Razorpay product selection and compliance readiness.

14. One-Sentence Product Definition

An AI-controlled student money manager that turns small recurring savings into an emergency-oriented cushion, adds community and merchant incentives, and lets students spend through explicit rules with Razorpay-backed payment execution.